

INNOMATICS RESEARCH LABS

RAG Internship Project

TECHNICAL DOCUMENTATION

RAG-Based Customer Support Assistant

Version 1.0 | 2025

1. Introduction

1.1 What is RAG (Retrieval-Augmented Generation)?

Retrieval-Augmented Generation (RAG) is an AI architecture pattern that enhances Large Language Models (LLMs) by grounding their responses in external, domain-specific knowledge. Standard LLMs are trained on static datasets and cannot access proprietary or up-to-date information. RAG solves this by dynamically retrieving relevant documents at inference time and providing them as context to the LLM.

The RAG pipeline consists of two primary phases:

- Indexing Phase (offline): Documents are loaded, chunked, embedded, and stored in a vector database.
- Inference Phase (online): A user query is embedded, similar documents are retrieved, and an LLM generates a contextually grounded response.

1.2 Why RAG is Needed

- LLMs hallucinate: Without grounding, LLMs generate plausible but incorrect answers. RAG constrains the LLM to only use verified knowledge.
- Domain knowledge: Organizations have proprietary knowledge (policies, manuals, FAQs) that public LLMs don't know. RAG makes this knowledge queryable.
- Cost efficiency: Fine-tuning LLMs on domain data costs thousands of dollars. RAG achieves similar accuracy at a fraction of the cost.
- Auditability: RAG provides source citations, enabling users to verify where an answer came from.
- Freshness: RAG can be updated by re-indexing documents without retraining the model.

1.3 Use Case Overview

This system implements a Customer Support Assistant for a product company. The knowledge base consists of PDF documents including product manuals, return policies, FAQ documents, and service agreements. When a customer submits a query, the system retrieves the most relevant sections from these documents and generates a precise, grounded answer. Complex or sensitive queries are escalated to human agents.

2. System Architecture Explanation

2.1 Detailed Explanation of HLD Components

Document Ingestion Pipeline

The ingestion pipeline is a one-time (or scheduled) batch process. PDFs are loaded using LangChain's PyPDFLoader, which handles encoding normalization and extracts text with page-level metadata. The text is then split using RecursiveCharacterTextSplitter with a chunk size of 800 characters and 100-character overlap. This overlap ensures that information spanning chunk boundaries is not lost.

Each chunk is then passed through the embedding model (sentence-transformers/all-MiniLM-L6-v2) to produce a 384-dimensional dense vector. These vectors, along with the original text and metadata, are stored in ChromaDB. ChromaDB builds an HNSW (Hierarchical Navigable Small World) index over the vectors for fast approximate nearest-neighbor search.

Query-Time Pipeline

At runtime, when a user submits a query, LangGraph initializes a new RAGState object. The input_node validates and preprocesses the query. The retrieve_node embeds the query using the same embedding model (critical for consistent vector space) and queries ChromaDB for the top-5 most similar chunks. The generate_node constructs a structured prompt and calls the LLM. The route_node evaluates confidence and decides whether to answer automatically or escalate.

2.2 Component Interactions

The components interact through the shared RAGState object in LangGraph. Each node reads from and writes to this state, creating a clean data pipeline:

- DocumentProcessor → ChunkingEngine → EmbeddingEngine → VectorStoreManager (ingestion)
- UserQuery → RAGState → input_node → retrieve_node → generate_node → route_node → output_node/hitl_node (runtime)
- ChromaDB is the central persistent store accessed by both the ingestion pipeline and the retrieval module
- LLM is called only once per query in the generate_node to minimize API costs

3. Design Decisions

3.1 Chunk Size Choice — 800 Characters

Chunk size selection involves a fundamental trade-off between retrieval precision and context completeness.

- Too small (< 200 chars): Chunks lack sufficient context for the LLM to generate meaningful answers. Retrieval may return too many fragments.
- Too large (> 1500 chars): Chunks may contain multiple topics, reducing retrieval precision. Larger chunks also consume more LLM context window.
- 800 characters: Empirically validated for knowledge base content. Captures 2-4 sentences — enough for a complete thought while remaining topic-focused.
- 100-char overlap: Preserves sentence continuity across chunk boundaries without significant redundancy.

3.2 Embedding Strategy — Local Sentence Transformers

The all-MiniLM-L6-v2 model was chosen over API-based embeddings for the following reasons:

- Zero API cost: Local inference eliminates per-token embedding costs (significant for large document sets)
- Data privacy: Sensitive company documents never leave the local environment
- Consistency: Embedding model version is pinned — no risk of API-side model updates changing vector space
- Speed: 50-100ms per batch of 32 chunks — acceptable for batch ingestion
- Quality: 384-dimension embeddings provide excellent semantic similarity for English text

3.3 Retrieval Approach — Cosine Similarity with Score Thresholding

Cosine similarity is preferred over dot product because it normalizes for vector magnitude, making it robust to varying text lengths. The score threshold of 0.35 was calibrated through testing on sample queries against the knowledge base. Scores below this threshold indicate the retrieved content is not genuinely relevant to the query, and escalation is more appropriate than forcing a low-quality answer.

3.4 Prompt Design Logic

```
SYSTEM_PROMPT = '''
You are a helpful customer support assistant for [Company Name].
Answer ONLY based on the provided context. Do NOT use prior knowledge.
If the context doesn't contain the answer, say: 'I don't have enough
information to answer this. Escalating to a human agent.'
Always cite the source document and page number.
'''

USER_PROMPT = f'''
Context:
{formatted_chunks}

Question: {query}

Answer:
```

'''

This prompt design enforces grounding (no hallucination), transparency (source citation), and graceful failure (explicit escalation signal when knowledge is insufficient).

4. Workflow Explanation

4.1 LangGraph Usage

LangGraph extends LangChain with a graph-based execution model. Unlike simple chain pipelines, LangGraph supports:

- **Stateful execution:** A typed state dictionary (RAGState) flows through all nodes, eliminating the need to pass data manually between functions.
- **Conditional edges:** Based on runtime values in the state, the graph can take different paths — enabling the auto-answer vs. HITL fork.
- **Interrupt/Resume:** The graph can pause at a designated node (hitl_node), await external input (human agent response), and resume — enabling true HITL without breaking the workflow abstraction.

4.2 Node Responsibilities

- **input_node:** Sanitizes query, removes PII if configured, sets initial state fields
- **retrieve_node:** The retrieval layer — embeds query, fetches top-k chunks, computes average similarity score
- **generate_node:** Core LLM call — builds structured prompt, calls LLM API, extracts answer and confidence
- **route_node:** Decision point — evaluates routing criteria, returns edge label ('output' or 'hitl')
- **output_node:** Formats response with source citations, computes latency, returns final QueryResult
- **hitl_node:** Escalation handler — serializes context, queues for human, awaits human_answer injection

4.3 State Transitions

The state evolves as it passes through each node. Key transitions:

- After **input_node**: query → processed_query (cleaned)
- After **retrieve_node**: retrieved_chunks and retrieval_scores populated
- After **generate_node**: llm_response and confidence set
- After **route_node**: routing decision made (no state change, only edge selection)
- After **output_node** OR **hitl_node**: final_answer set, escalated flag updated

5. Conditional Logic

5.1 Intent Detection

Intent detection operates at two levels:

Keyword-based Pre-Classification (fast, runs before retrieval):

- Complaint/escalation keywords: 'refund', 'complaint', 'legal', 'urgent', 'escalate' → direct to HITL
- Factual query keywords: 'what', 'how', 'when', 'where', 'policy' → standard RAG flow

LLM-based Confidence Post-Classification (after generation):

The LLM is prompted to output a confidence score (0.0-1.0) alongside its answer. This self-reported confidence, combined with retrieval scores, forms the final routing confidence metric.

5.2 Routing Decisions

The routing function implements a multi-criteria decision:

```
def routing_function(state: RAGState) -> Literal['output', 'hitl']:
    conf = state['confidence']
    scores = state['retrieval_scores']
    chunks = state['retrieved_chunks']
    avg_score = sum(scores) / max(len(scores), 1)

    if not chunks:
        return 'hitl' # No knowledge found
    if avg_score < 0.35:
        return 'hitl' # Weak retrieval
    if conf < 0.40:
        return 'hitl' # LLM uncertainty
    return 'output' # All criteria met
```

6. HITL Implementation

6.1 Role of Human Intervention

The Human-in-the-Loop mechanism serves three critical roles in this system:

- Quality Gate: Prevents low-confidence AI answers from reaching customers, protecting brand reputation
- Edge Case Handler: Captures queries that fall outside the knowledge base, informing future document updates
- Trust Builder: Ensures customers dealing with sensitive issues (refunds, complaints) receive human attention

6.2 Benefits and Limitations

Aspect	Benefit	Limitation
Quality	High-quality answers for complex queries	Adds latency (hours vs seconds)
Trust	Customers feel heard for sensitive issues	Increases operational cost
Learning	Escalated Q&A pairs can train future models	Requires structured data capture
Coverage	No query goes unanswered	Scales poorly without agent staffing
Auditability	Full audit trail of human decisions	Privacy implications of data retention

7. Challenges & Trade-offs

7.1 Retrieval Accuracy vs. Speed

Higher retrieval accuracy can be achieved by increasing k (more chunks retrieved) and using a larger, more powerful embedding model (e.g., text-embedding-3-large). However, this increases latency significantly. The chosen configuration (k=5, MiniLM-L6-v2) balances accuracy and speed for the target P95 < 3s SLA. For higher accuracy requirements, re-ranking with a cross-encoder model can be applied post-retrieval.

7.2 Chunk Size vs. Context Quality

Smaller chunks (200-300 chars) yield more precise retrieval — the retrieved content is tightly focused on the query topic. However, they often lack sufficient context for the LLM to generate a complete answer. Larger chunks (1000-1500 chars) provide richer context but reduce retrieval precision (a chunk may contain both relevant and irrelevant content). The 800-char setting with 100-char overlap is the empirically validated middle ground for customer support content.

7.3 Cost vs. Performance

Configuration	Cost/Month (est.)	Performance
Local Embed + GPT-4o-mini	\$5-15	Good accuracy, fast
OpenAI Embed + GPT-4o	\$50-200	High accuracy, slower
Local Embed + Local LLM (Ollama)	\$0	Moderate accuracy, requires GPU
OpenAI Embed + Claude 3.5 Sonnet	\$30-150	Excellent accuracy, fast

The recommended production configuration is Local Embedding + GPT-4o-mini, providing the best cost-to-performance ratio for a customer support workload.

8. Testing Strategy

8.1 Testing Approach

Unit Tests:

- DocumentProcessor: test with valid PDF, corrupt PDF, empty PDF
- ChunkingEngine: verify chunk sizes stay within bounds, overlap correctness
- RetrieverEngine: mock ChromaDB, test score thresholding logic
- routing_function: parametrized tests for all routing conditions

Integration Tests:

- Full pipeline test: PDF → Chunks → Embeddings → ChromaDB → Query → Answer
- Graph execution test: verify state transitions at each node
- HITL flow test: trigger escalation, inject human answer, verify resume

Evaluation (RAG-specific):

- Faithfulness: Is the answer grounded in the retrieved chunks? (Target > 90%)
- Answer Relevance: Does the answer address the query? (Target > 85%)
- Context Recall: Did retrieval capture the relevant chunks? (Target > 80%)

8.2 Sample Queries for Testing

Query	Expected Route	Test Goal
What is your return policy?	output	Happy path — well-covered topic
How do I reset my password?	output	Technical FAQ — direct answer
I want to sue your company	hitl	Sensitive keyword escalation
What is the meaning of life?	hitl	Out-of-domain query
My order #12345 is missing	hitl	No context available in KB

9. Future Enhancements

9.1 Multi-Document Support

The current implementation supports single or multiple PDFs loaded at startup. Future enhancement: real-time document ingestion API that accepts new PDFs via webhook, automatically chunks, embeds, and adds to ChromaDB without system restart. This enables the knowledge base to grow dynamically as new product documentation is published.

9.2 Feedback Loop

Implement a thumbs up/down rating on each response. Negative feedback triggers automatic re-routing of the query to HITL for human review. High-quality human-verified Q&A pairs are stored in a feedback dataset and periodically used to fine-tune the routing thresholds or the LLM itself, creating a self-improving system.

9.3 Memory Integration

Integrate conversation memory using LangGraph's built-in checkpointing. Each conversation thread maintains a summary of prior exchanges, enabling the system to answer follow-up questions correctly. For example: 'Tell me about returns' followed by 'How long does it take?' correctly interprets 'it' as 'the return process'.

9.4 Deployment

Production deployment path:

- Containerize with Docker: separate containers for API server, embedding service, and ChromaDB
- Deploy on AWS ECS or Google Cloud Run for auto-scaling
- Replace ChromaDB with Pinecone or Weaviate for managed vector storage at scale
- Add monitoring: LangSmith for LLM tracing, Prometheus + Grafana for system metrics
- CI/CD pipeline: automated testing on PDF ingestion and query regression suite